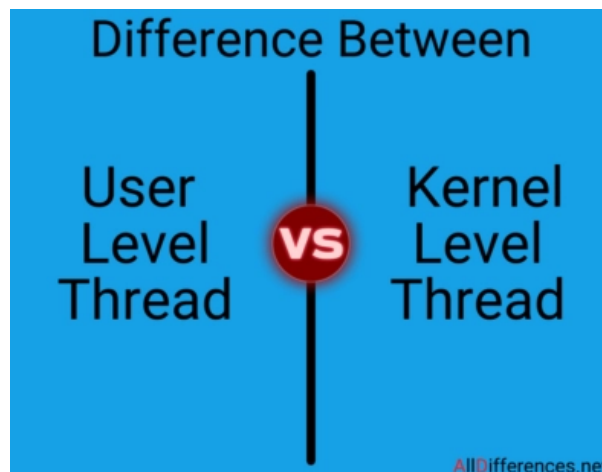


Difference between User Level and Kernel Level Thread

Difference between User Level and Kernel Level Thread in Tabular Form

[Contents \[show\]](#)

Summary: The Major **Difference between User Level and Kernel Level Thread** is that **User Level Threads** is managed by the User. **Kernel Level Threads** managed by **Operating System**. All modern operating systems support the threading model. Implementation of a thread will change according to the operating system.



Comparison Chart

User Level Thread

User-level threads are faster to create and manage.

Kernel Level Thread

Kernel level threads are slower to create and manage.

User Level Thread	Kernel Level Thread
Implemented by a thread_library user level.	Operating system support directly to Kernel threads.
User level thread can run on any operating system.	Kernel level threads are specific to .the operating system.
Support provided at the user level called user-level thread	Support may be provided by Kernel is called Kernel level threads.
Multithread applications cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.
Implementation of User threads is easy.	Implementation of Kernel thread is complicated.
User level threads are also called many to one mapping thread.	Kernel level thread support one to one thread mapping.
Context switch time is less.	Context switch time is more.
Context switch requires no hardware support.	Hardware support is needed.
Example: User-thread libraries include POSIX Pthreads, Mach C-threads, and Solaris 2 UI-threads.	Example: Windows NT, Windows 2000, Solaris 2, BeOS, and Tru64 UNIX (formerly Digital UNIX)-support kernel threads.

User Level Thread

- User level thread uses user space for thread scheduling. These threads are **transparent** to the operating system.
- User level threads are created by runtime libraries that cannot execute privileged instructions.
- User level threads have low overhead but it can achieve high -performance in computation.
- User level threads are managed entirely by the run-time system.
- User level threads are small and faster. A thread is simply represented by a PC, registers, stack and small thread control block.
- The code for creating and destroying the thread, message passing, and data transfer, thread scheduling is included into thread library. A kernel is unaware of user-level thread.
- User level threads do not invoke the Kernel for scheduling decision.
- User level thread is also called **many to one mapping thread** because the operating system maps all threads in a multithreaded process to a single execution context. The operating system considers each multithreaded processes as a single execution unit.
- **Example:** POSIX Pthreads and Mach C-threads.

User Level Thread Advantages

- Kernel-mode privilege does not require for thread switching.
- These threads are fast to create and manage.
- User level thread works even if the OS does not support threads.
- User level threads are more portable.
- Threading library controls the **flow of thread**.

User Level Thread Disadvantages

- If thread blocks, the Kernel may block all threads

- Not suitable for the multiprocessor system.
- User level threads also do not support system-wide scheduling priority.

Kernel Level Thread

- In Kernel level thread, thread management is done by Kernel. Operating systems support the Kernel level thread.
- Since Kernel managing threads, Kernel can schedule another thread if a given thread blocks rather than blocking the entire processes.
- Kernel level thread support one to one thread mapping. This mapping requires each user thread with kernel thread. Operating system performs this mapping.
- Threads are constructed and controlled by system calls. The system knows the state of each thread.
- Thread management code is not included in the application code. It is the only API to the Kernel thread. Windows operating system uses this facility.
- Any application can be programmed to be multithreaded. All of the threads within an application are supported within a single process.
- Kernel performs scheduling on a thread basis. The Kernel support and management, thread creation only in Kernel space.
- Kernel level threads are slower than user level threads.
- Example: Windows 95/99/NT. Sun Solaris and Digital UNIX.

Kernel Level Thread Advantages

- Each thread can be treated separately.
- A thread blocking in the Kernel does not block all other threads in the same process.
- Kernel routines itself as multithreaded.

Kernel Level Thread Disadvantages

- Slower than the user level thread.
- There will be overhead and increased in Kernel complexity.

More Differences

- [Difference Between Monolithic Kernel and Microkernel](#)
- [Difference Between Hard and Soft Real-Time System](#)



Categories

- Computer Engineering
- DBMS
- Electrical Engineering
- Mechanical Engineering
- NETWORKING
- OPERATING SYSTEM
- PROGRAMMING
- SOFTWARE

Ads

Recent Posts

Difference Between Greedy and Dynamic Programming

Difference between Sequential Search and Binary Search

Difference between Spring and Shock Absorber

Difference between Hotchkiss Drive and Torque Tube Drive

Difference between Fluid Coupling and Torque Converter

Copyright © 2022 All Differences

 CONTACT US

PRIVACY POLICY